GGIB Part 3 - Formal Scoring Framework

Authors: Clarence Downs, Caroline Brooks
Status: Draft 0.2 (revised per C. Brooks architectural review)
Created: 2026-03-18
Revised: 2026-03-23
Parent: AGI-Benchmark-Blueprint
Depends on: GGIB-Part1-Critical-Analysis, GGIB Part 2 - Metacognition Task Family Design
Informed by: Metacognition-Benchmark-Critique, Metacognition-Benchmark-Synthesis-Report, Orchestrator_Epistemic_Capture_v2
Competition: Kaggle - Measuring Progress Toward AGI: Cognitive Abilities
Track: Metacognition
Normative standard: Ideal epistemic agent (locked in Part 1)

Revision Notes (v0.2)

This revision incorporates architectural feedback from Caroline Brooks on the matcher calibration vulnerability identified in v0.1. The core change: Dimension 2 (Reasoning Specificity) now uses a **dual-path validation architecture** that separates reference presence from reference utilization. This closes a single-point-of-anchoring vulnerability that v0.1 left open inside a system otherwise designed to avoid single-point collapse.

Specific changes from v0.1:

- Dimension 2 (RS) redesigned with dual-path scoring (Reference + Utilization)
- Matcher calibration section expanded from a footnote to a full architectural section

- Tiered calibration replaces global threshold
- Contrastive grading added as a defense against shared representational bias
- Matcher drift monitoring added as a runtime integrity check
- Pseudocode updated to reflect new RS scoring function

All other dimensions, composite scores, and worked examples remain structurally unchanged. Worked examples are re-scored under the new RS formula.

Design Constraints Carried Forward

From GGIB-Part1-Critical-Analysis and Metacognition-Benchmark-Critique:

- All scoring dimensions must be **computable functions** over structured model outputs, not subjective rubrics.
- The scoring system must **discriminate** between appropriate epistemic behavior and epistemic theater.
- Scoring must **not collapse** into answer accuracy, raw confidence calibration, or hedge-word detection.
- The system must **penalize** both overconfidence and blanket hedging.
- **Repeated-trial stability** must be assessed with a named statistical method.
- Every design choice must state an alternative considered and why it was rejected.

From Metacognition-Benchmark-Synthesis-Report:

- Scoring must operate on **behavioral traces**, not single-point outputs.
- The architecture is: `task -> response -> behavioral trace -> epistemic classification -> score`.

From Orchestrator_Epistemic_Capture_v2:

- **The meta-evaluation principle:** A system cannot score itself. The scoring framework cannot rely on the model's own confidence claims as the primary signal. It requires external anchoring: ground truth evidence conditions defined independently of the model's output.
- **KL-divergence as drift detection:** The same mathematical tool used for shadow policy comparison can measure whether a model's epistemic behavior is stable or stochastic across repeated runs.
- **Federated capture risk in scoring:** If multiple scoring dimensions rely on the same detection method, they share an epistemic frame and can all be gamed simultaneously. Scoring dimensions must be epistemically diverse, each dimension must rely on a structurally distinct detection mechanism.

From C. Brooks architectural review (2026-03-23):

- **Single-point anchoring vulnerability:** The matcher function underlying RS and MEI acts as a truth proxy for evidence engagement. If that function drifts, both dimensions degrade simultaneously, reintroducing the single-point collapse the architecture was designed to prevent.
- **Dual-path validation:** Reference presence alone is insufficient. The matcher must also assess reference utilization: whether the referenced element is functionally used in reasoning, not merely mentioned.
- **Shared representational bias:** If the LLM grader shares training distribution with the evaluated model, it will systematically over-credit "legible reasoning." Contrastive grading mitigates this.
- **Tiered calibration:** A single global matcher threshold across structurally different task families is too coarse. Family-specific and domain-sensitive thresholds are required.

Scoring Architecture

The Problem the Scoring System Must Solve

The scoring system must answer a question that ordinary benchmarks do not ask:

> Given the actual state of the evidence, was the system's epistemic behavior appropriate?

This is not:

- "Was the answer correct?" (performance)
- "Was the confidence number well-calibrated?" (calibration)
- "Did the system use uncertainty language?" (keyword detection)

It is:

- "Did the system's observable behavior covary with the evidence condition in a pattern consistent with an ideal epistemic agent?"

Why a Single Score Is Insufficient

A single metacognition score collapses the same way a single AGI score does, it hides jaggedness. A model can excel at contradiction detection while being terrible at premature closure resistance. A single score averages these into mediocrity and loses the diagnostic signal.

The scoring system produces **five dimension scores** plus **two composite scores** plus **one stability score**.

Alternative rejected: A weighted single score. Rejected because weighting implicitly ranks metacognitive behaviors, and no defensible theoretical basis exists for ranking (e.g.) contradiction detection as more important than confidence calibration. Profiles are more honest than rankings.

Five Scoring Dimensions

Each dimension is a computable function over structured model outputs and labeled task metadata. Each uses a structurally distinct detection mechanism to prevent federated scoring capture.

Dimension 1: Epistemic Judgment Accuracy (EJA)

What it measures: Whether the system's top-level epistemic judgment (answerable/unanswerable, contradiction present/absent, evidence sufficient/insufficient) matches the ground truth evidence condition.

Detection mechanism: Direct comparison of categorical output fields against labeled ground truth.

Computable definition:

```
For each task instance i:

  EJA_i = 1  if epistemic_judgment_i == ground_truth_condition_i
  EJA_i = 0  otherwise

EJA = mean(EJA_i) across all instances
```

Where `epistemic_judgment` is the structured output field:

- Family 1 (EGD): `answerable` field
- Family 2 (CDSP): `contradiction_detected` field
- Family 3 (PCR): `sufficient_evidence` field
- Family 4 (SEI): `revised` field (against labeled correct revision behavior)
- Family 5 (CCED): `evidence_quality` field (within +/-1 ordinal level of labeled quality)

Why this is not answer accuracy: EJA measures the epistemic judgment about the evidence state, not whether the final answer is correct. A system that gives the right answer but misjudges the evidence condition (e.g., says "answerable" when evidence is insufficient, but happens to guess correctly) scores EJA = 0 on that instance.

Range: [0, 1]. Higher is better.

Dimension 2: Reasoning Specificity (RS) - Dual-Path Architecture

What it measures: Whether the system's reasoning demonstrates genuine evidence engagement, not merely surface-level mention of evidence elements.

Detection mechanism: Dual-path evidence-element analysis. This is structurally distinct from Dimension 1's categorical comparison. It operates on the free-text reasoning field through two independent validation paths.

v0.2 design rationale (per C. Brooks review): v0.1 used a single-path matcher that answered "Does the reasoning reference element X?" This created two vulnerabilities. First, a model could game RS by listing evidence elements without using them in reasoning. Second, the matcher became a single point of epistemic anchoring: if it drifted, both RS and MEI degraded simultaneously. The dual-path architecture separates reference detection from utilization detection, requiring the model to both mention and functionally use evidence elements.

Path A: Reference Presence (R)

Does the reasoning mention or paraphrase the key evidence element?

This is the original RS check from v0.1. It detects whether the evidence element appears in the reasoning at all.

```
For each task instance i:

  Let K_i = set of key evidence elements (labeled, typically 3-8
per task)
  Let R_i = set of evidence elements referenced in the model's
REASONING field

  R_score_i = |R_i ∩ K_i| / |K_i|
```

Path B: Reference Utilization (U)

Is the referenced element functionally used in the reasoning chain?

"Functionally used" means the element is transformed, compared, linked to the answerability constraint, or identified as the cause of the evidence gap. Mere mention does not qualify.

Examples of utilization:

- "The reactant quantities are not provided, making stoichiometric calculation impossible" - the missing quantity is linked to the impossibility of answering. This is utilization.
- "The 450 MPa value from Source 1 directly conflicts with the 312 MPa value from Source 2, and this discrepancy exceeds measurement noise" - the numeric values are compared and the comparison drives the contradiction judgment. This is utilization.

Examples of mention without utilization:

- "The evidence mentions 450 MPa tensile strength" - the value is restated but not used in reasoning. This is reference only.
- "The missing reactant quantities, temperatures, and pressures were noted" - elements are listed but not connected to the epistemic judgment. This is reference only.

```
For each task instance i:
```

```
  Let K_i = set of key evidence elements
  Let U_i = set of evidence elements functionally utilized in
reasoning

  U_score_i = |U_i ∩ K_i| / |K_i|
```

Detection of utilization: Each key evidence element has a labeled **utilization pattern**: how an ideal epistemic agent would use that element in reasoning. Detection checks whether the reasoning field contains a transformation, comparison, causal link, or constraint derivation involving the element, not just a restatement.

Implementation paths:

- **LLM-graded (recommended for utilization):** A grading prompt that asks "Does this reasoning use [element X] to support or constrain the epistemic judgment, or does it merely mention it? Answer: used / mentioned / absent."
- **Embedding-based (acceptable for reference only):** Cosine similarity check between element and reasoning sentences.

Combined RS Score

```
RS_i = lambda_1 * R_score_i + lambda_2 * U_score_i

Where lambda_1 = 0.30, lambda_2 = 0.70
```

Utilization carries more than twice the weight of reference. This is the core anti-theater mechanism. A model that lists evidence elements without using them scores at most RS = 0.30 *R_score. A model that uses elements in its reasoning chain scores up to RS = 0.30* R_score + 0.70 * U_score.

Why lambda_2 > lambda_1: Reference presence alone is gameable. A model can learn to echo evidence elements without processing them. Utilization requires the model to demonstrate that the evidence element played a functional role in the reasoning chain. This is harder to fake because it requires generating reasoning that is structurally connected to specific evidence, not just lexically adjacent to it.

Alternative rejected (v0.1 approach): Single-path RS using reference presence only. Rejected after C. Brooks review identified that this allows keyword anchoring strategies where the model mentions elements without engaging them, and that it creates a single point of epistemic anchoring where the matcher becomes the weakest link in the scoring architecture.

Range: [0, 1]. Higher is better.

Dimension 3: Confidence Appropriateness (CA)

What it measures: Whether the system's stated confidence is proportional to the evidence quality, not whether the confidence number is well-calibrated against accuracy.

Detection mechanism: Deviation scoring against evidence-condition-defined confidence ranges. Structurally distinct from Dimensions 1 and 2, this operates on the numeric confidence field against labeled appropriate ranges, not on categorical judgments or free-text reasoning.

Computable definition:

Each task instance has a labeled **appropriate confidence range** [C_low, C_high] defined by the evidence condition:

Evidence Condition	Appropriate Confidence Range
Sufficient evidence, clear answer	[75, 100]
Moderate evidence, reasonable inference	[50, 80]
Insufficient evidence	[0, 20]
Contradictory evidence	[15, 45]

Evidence Condition	Appropriate Confidence Range
Misleading abundance (premature closure trap)	[20, 50]
Non-diagnostic new evidence (should not change)	[C_turn1 - 10, C_turn1 + 10]

```
For each task instance i:

  Let C_i = model's stated confidence
  Let [C_low_i, C_high_i] = labeled appropriate range

  If C_low_i <= C_i <= C_high_i:
    CA_i = 1.0

  If C_i > C_high_i:
    CA_i = max(0, 1.0 - alpha * (C_i - C_high_i) / 100)
    # Overconfidence penalty, scaled by distance from range

  If C_i < C_low_i:
    CA_i = max(0, 1.0 - beta * (C_low_i - C_i) / 100)
    # Under-confidence penalty, scaled by distance from range
```

Asymmetric penalty rates: alpha > beta. Overconfidence is penalized more heavily than under-confidence, because overconfidence under insufficient evidence is the primary metacognitive failure mode the benchmark targets. Recommended: alpha = 2.0, beta = 1.5.

Alternative rejected: Expected Calibration Error (ECE) across accuracy bins. Rejected because ECE measures confidence-accuracy correlation, which is a performance metric, not an epistemic one. A model can be well-calibrated on ECE by learning difficulty cues rather than monitoring its reasoning. CA measures confidence-evidence correlation, which requires processing the actual evidence body.

Why this catches blanket hedging: A model that always outputs CONFIDENCE: 15 scores well on insufficient-evidence tasks (within range) but poorly on sufficient-evidence tasks (far below the [75, 100] range). The penalty accumulates across the full task set. Blanket hedging cannot achieve a high aggregate CA score.

Range: [0, 1]. Higher is better.

Dimension 4: Revision Appropriateness (RA)

What it measures: Whether the system revises when it should and holds steady when it shouldn't, specifically in multi-turn interactions (Task Family 4: SEI).

Detection mechanism: Behavioral state-change analysis across turns. Structurally distinct from all other dimensions, this compares model outputs at Turn 1 and Turn 2 and classifies the transition pattern.

Computable definition:

For each SEI task instance, the system produces a Turn 1 response and a Turn 2 response after receiving new evidence. The evidence is labeled as one of three types: `disconfirming`, `confirming`, `non_diagnostic`.

```
For each SEI task instance i:

  Let evidence_type_i in {disconfirming, confirming,
non_diagnostic}
  Let revised_i = model's REVISED field at Turn 2
  Let delta_confidence_i = |confidence_turn2 - confidence_turn1|
  Let trigger_specific_i = 1 if REVISION_TRIGGER cites specific
new evidence, 0 otherwise
```

```
  # Core revision accuracy
  revision_correct_i:
    if evidence_type == disconfirming:  correct = (revised ==
yes)
    if evidence_type == confirming:     correct = (revised ==
no)
    if evidence_type == non_diagnostic: correct = (revised ==
no)

  # Revision trigger specificity (only scored when revision
occurred)
  trigger_score_i:
    if revised == yes AND evidence_type == disconfirming:
      trigger_score = trigger_specific_i  # Should cite specific
new evidence
    if revised == yes AND evidence_type != disconfirming:
      trigger_score = 0  # Revised when shouldn't have, no
credit for trigger

  # Confidence stability (only scored for non-diagnostic
evidence)
  stability_score_i:
    if evidence_type == non_diagnostic:
      stability_score = max(0, 1.0 - delta_confidence_i / 30)
      # Confidence shifts > 30 points on irrelevant evidence ->
score 0
    else:
      stability_score = N/A (not scored)

  RA_i = weighted combination:
    0.50 * revision_correct_i
    + 0.30 * trigger_score_i (when applicable)
    + 0.20 * stability_score_i (when applicable)
```

Why this catches performative revision: A model that revises every time new evidence appears (regardless of type) scores revision_correct = 1 on disconfirming tasks but 0 on confirming and non-diagnostic tasks. Aggregate RA

will be ~0.50 at best. A model that genuinely discriminates evidence types scores near 1.0 across all three conditions.

Why trigger specificity matters: A model that revises with "Based on the new information, I'm updating my assessment" scores trigger_specific = 0. A model that revises with "The finding that 85% of tickets come from non-upgraded users directly undermines my initial hypothesis" scores trigger_specific = 1. This is the theater detection layer within the revision dimension.

Only applies to: Task Family 4 (SEI). For task families without multi-turn interaction, RA is not computed and does not contribute to the composite score.

Range: [0, 1]. Higher is better.

Dimension 5: Missing Element Identification (MEI)

What it measures: Whether the system identifies specific missing evidence elements or missing decision factors when the evidence is incomplete.

Detection mechanism: Set comparison between identified missing elements and labeled missing element lists. Structurally distinct from all other dimensions, this operates on list-type output fields against labeled element sets.

Computable definition:

Applies to Task Families 1 (EGD) and 3 (PCR), where the evidence is insufficient or where the evidence creates a premature closure trap.

```
For each applicable task instance i:

  Let M_i = labeled set of missing evidence elements (ground
truth, typically 2-6)
  Let S_i = set of elements the model listed as missing
```

```
  # Precision: of what the model listed, how many are actually
missing?
  precision_i = |S_i ∩ M_i| / |S_i|  (0 if |S_i| = 0)

  # Recall: of what is actually missing, how many did the model
find?
  recall_i = |S_i ∩ M_i| / |M_i|

  # F1 score as the MEI score
  MEI_i = 2 ∗ (precision_i ∗ recall_i) / (precision_i +
recall_i)
          (0 if both precision and recall are 0)
```

Why precision matters: A model that dumps a long generic checklist ("cost, risk, timeline, regulatory, stakeholder, environmental, legal, capacity, maintenance...") will have high recall but low precision. The F1 score punishes this.

Why recall matters: A model that identifies one genuine missing element but misses four others gets credit for what it found but is penalized for the gaps.

Why this catches theater: "More information would be needed" with an empty or generic MISSING field scores MEI = 0. The model performed the act of identifying insufficiency (caught by Dimension 1: EJA) but did not demonstrate evidence-sensitive reasoning about what specifically is missing.

Matching method for set comparison: Each labeled missing element is defined as a semantic unit. The model's listed items are matched to labeled elements using the reference presence path (Path A) of the matcher system, with the same tiered calibration requirements described in the Matcher Architecture section below.

Only applies to: Task Families 1 (EGD) and 3 (PCR). For other families, MEI is not computed.

Range: [0, 1]. Higher is better.

Epistemic Diversity Check Across Dimensions

Per the federated capture principle from Orchestrator_Epistemic_Capture_v2, the five dimensions must use structurally distinct detection mechanisms. If they share a method, they share a vulnerability.

Dimension	Primary Detection Mechanism	Input Type
EJA (Epistemic Judgment Accuracy)	Categorical match against labeled ground truth	Categorical output fields
RS (Reasoning Specificity)	Dual-path evidence-element analysis (reference + utilization)	Free-text reasoning field
CA (Confidence Appropriateness)	Numeric range deviation scoring	Numeric confidence field
RA (Revision Appropriateness)	Cross-turn behavioral state-change analysis	Multi-turn output comparison
MEI (Missing Element Identification)	Set-comparison (F1) against labeled missing elements	List-type output fields

Verification: No two dimensions use the same detection mechanism. No two dimensions operate on the same output field type. A gaming strategy that defeats one dimension (e.g., producing correct categorical judgments) does not automatically defeat another (e.g., producing reasoning that functionally uses evidence elements). The dimensions are epistemically diverse.

v0.2 note: The dual-path architecture in RS also reduces the coupling between RS and MEI. In v0.1, both used the same single-path matcher, creating a shared vulnerability. In v0.2, RS uses the full dual-path system (reference + utilization) while MEI uses only the reference presence path (Path A). They now share a component rather than a complete mechanism, and the component they share (Path A) is the simpler, more robust path.

Matcher Architecture (v0.2)

This section replaces the v0.1 footnote on matcher calibration. It is now a first-class architectural component of the scoring system.

The Problem

The matcher function underlies both RS (Dimension 2) and MEI (Dimension 5). In v0.1, C. Brooks identified this as a single point of epistemic anchoring:

> "You have a single point of epistemic anchoring inside a system designed to avoid single-point collapse."

The matcher is doing more than string matching. It is acting as a truth proxy for evidence engagement. If it drifts, RS collapses into noise and MEI becomes gameable via structured over-generation. Because RS carries high weight in ECS, the entire benchmark loses discrimination power.

Dual-Path Validation

The matcher system now has two independent validation paths:

Path A: Reference Presence

- Determines whether the reasoning text mentions or paraphrases a key evidence element
- Implementation: embedding-similarity check (cosine similarity > family-specific threshold)
- This is the simpler, more robust path
- Used by both RS and MEI

Path B: Reference Utilization

- Determines whether the referenced element is functionally used in the reasoning chain

- "Functionally used" means: transformed, compared, linked to the answerability constraint, or identified as the cause of the evidence gap
- Implementation: LLM-graded with a structured prompt (see below)
- This is the more discriminating path
- Used by RS only (not MEI)

Contrastive Grading (Path B)

Per C. Brooks' feedback on shared representational bias, the LLM grading for Path B uses contrastive grading rather than absolute grading.

Absolute grading (rejected):

> "Does this reasoning use element X? Yes or no."

This is vulnerable to grader bias: the grading model may systematically over-credit reasoning that is "legible" in its own style.

Contrastive grading (adopted):

> "Here are two reasoning excerpts about the same evidence. Which one functionally uses [element X] to support or constrain the epistemic judgment? Excerpt A uses it, Excerpt B merely mentions it, or both/neither."

Contrastive grading forces the grader to discriminate between utilization and mention, reducing the risk that legibility alone drives the score.

Calibration of the contrastive grader:

- Build a calibration set of 50 contrastive pairs (one using the element, one merely mentioning it) with human-annotated gold labels
- Run the grading model on the calibration set
- Compute accuracy against human labels
- Minimum threshold: 85% agreement with human annotations
- If threshold is not met, revise the grading prompt or switch grading models

Tiered Calibration

A single global matcher threshold is too coarse. Different task families and domains produce different reasoning textures.

Tier 1: Global baseline threshold
Set from the full pilot calibration dataset. This is the default when family-specific data is insufficient.

Tier 2: Family-specific adjustment
EGD tasks (chemistry, physics) produce more precise, quantitative reasoning. PCR tasks (decision-making, facility siting) produce more discursive reasoning. SEI tasks produce comparative reasoning across turns. Each family gets its own calibrated threshold, computed from the family-specific subset of the pilot calibration dataset.

Tier 3: Domain sensitivity flag
Within a family, quantitative domains (chemistry, physics, economics) and qualitative domains (policy, hiring, system architecture) may require different thresholds. If pilot data shows that domain type predicts matcher accuracy with > 10% variation, domain-specific thresholds are applied.

Calibration protocol:

1. Collect 50+ human reasoning responses per family during pilot (see GGIB Part 4 - Human Baseline Design)
2. Two independent raters annotate each response for reference presence (Path A) and reference utilization (Path B) per key evidence element
3. Compute Cohen's kappa per family. Target: kappa >= 0.75
4. Set family-specific embedding thresholds at the point that maximizes F1 against human annotations
5. Validate contrastive grader accuracy per family. Target: 85% agreement
6. If a family's kappa falls below 0.70, revise element definitions for that family before deployment

Matcher Drift Monitoring

The matcher is not calibrated once and left static. It must be monitored for drift during deployment.

Drift signal: If RS variance increases across runs without a corresponding change in EJA, the matcher may be degrading. The evidence judgments are stable (EJA consistent) but the reasoning scoring is becoming noisy (RS varying), which indicates the matcher is losing calibration.

Detection method:

- Track RS standard deviation across repeated runs on the same task set
- Track the correlation between RS and EJA across runs
- If RS standard deviation increases by > 20% between evaluation cycles, or if the RS-EJA correlation drops below 0.50, flag for re-calibration

Response: When drift is detected, re-run the matcher against the pilot calibration set. If accuracy has dropped, re-calibrate thresholds. If accuracy is stable but RS variance has increased, the issue may be in the model's reasoning variation rather than the matcher, which is a valid finding about the model, not a matcher failure.

Two Composite Scores

Composite A: Epistemic Conduct Score (ECS)

The primary composite score. Combines the five dimensions into a per-family and overall score.

```
For task families where all 5 dimensions are applicable (Family
4: SEI):
  ECS = 0.25 * EJA + 0.25 * RS + 0.20 * CA + 0.20 * RA + 0.10 *
MEI

For task families with 4 applicable dimensions (Families 1 and
3):
  ECS = 0.30 * EJA + 0.30 * RS + 0.25 * CA + 0.15 * MEI
```

```
For task families with 3 applicable dimensions (Families 2 and
5):
  ECS = 0.35 * EJA + 0.35 * RS + 0.30 * CA
```

Weights are higher for EJA and RS because these are the two most theater-resistant dimensions. EJA catches wrong epistemic judgments. RS (with dual-path architecture) catches right judgments with empty or non-functional reasoning. Together they form the core discrimination pair.

Alternative rejected: Equal weighting across all dimensions. Rejected because equal weighting gives disproportionate influence to dimensions that are easier to game (CA is more gameable than RS because a model can learn appropriate confidence ranges without processing evidence).

Composite B: Theater Detection Score (TDS)

A diagnostic score that specifically measures the gap between surface compliance and demonstrated evidence sensitivity. This score is not additive with ECS, it is a separate diagnostic.

```
TDS = EJA - RS

Interpretation:
  TDS near 0: The model's epistemic judgments and its reasoning
specificity are aligned.
              Judgments are backed by evidence-sensitive
reasoning.
  TDS > 0.2:  The model is making correct epistemic judgments
but its reasoning
              is vague or generic. High probability of
epistemic theater.
  TDS < 0:    The model's reasoning is specific but its top-
level judgments are wrong.
              This is unusual and may indicate a scoring
calibration issue.
```

Why this matters: A model with EJA = 0.90 and RS = 0.35 looks good on epistemic judgment accuracy but is likely performing theater, it learned the correct categorical responses without processing the evidence. TDS = 0.55 flags this. A model with EJA = 0.85 and RS = 0.80 shows aligned performance, its judgments are backed by specific reasoning. TDS = 0.05.

v0.2 note: Under the dual-path RS architecture, TDS becomes a stronger signal. Because RS now requires functional utilization of evidence (not just mention), a high TDS indicates not only that the model is producing vague reasoning, but that whatever evidence it does mention is not being used to drive its epistemic judgment. This is a more precise theater indicator than the v0.1 version.

Stability Score: Shannon Entropy Across Repeated Runs

From Orchestrator_Epistemic_Capture_v2, KL-divergence measures distributional drift between two decision-makers. Applied here, we measure whether a model's epistemic behavior is stable or stochastic across repeated runs of the same task set.

Method

1. Run the full benchmark on the same model K times (recommended: K >= 5).
2. For each task instance, record the model's epistemic judgment (the categorical output field used by Dimension 1).
3. For each task instance, compute the empirical distribution of epistemic judgments across K runs.
4. Compare this distribution to the "ideal stable" distribution (the same judgment every time = zero entropy).

Computable Definition

```
For each task instance i, across K runs:

  Let p_i(j) = proportion of runs where the model gave judgment
j
  Let H_i = Shannon entropy of p_i

  H_i = - SUM_j p_i(j) * log2(p_i(j))

  For binary judgments (answerable/not answerable):
    H_max = 1.0 (bits)
    Perfect stability: H_i = 0 (same answer every run)
    Maximum instability: H_i = 1.0 (50/50 split)

  Stability_i = 1.0 - (H_i / H_max)
```

Aggregate Stability Score:

```
Stability = mean(Stability_i) across all instances
```

Interpretation:

- Stability > 0.90: Highly consistent epistemic behavior across runs.
- Stability 0.70-0.90: Moderate consistency. Some stochastic variation.
- Stability < 0.70: Unreliable epistemic behavior. Judgments are substantially influenced by sampling randomness.

Why entropy rather than KL-divergence for this specific application: KL-divergence requires two distributions to compare. In the orchestrator paper, the two distributions are the live orchestrator and the shadow policy. Here, there is no "shadow model". We are measuring a single model's self-consistency. Shannon entropy directly measures the dispersion of a single distribution. However, if comparing two models' stability profiles, KL-divergence between their response distributions on the same task set is the appropriate metric.

Alternative rejected: Simple agreement rate (percentage of runs with the same answer). Rejected because agreement rate treats all disagreements equally.

Entropy captures the shape of disagreement, a 4/1 split is more stable than a 3/2 split, and entropy reflects this while agreement rate does not.

Penalty and Reward Logic

Penalty Structure

The scoring system has three explicit penalty mechanisms:

Penalty 1: Overconfidence Penalty

Applied within Dimension 3 (CA) via the asymmetric alpha coefficient. A model that states CONFIDENCE: 85 under insufficient evidence (appropriate range [0, 20]) receives:

```
CA_i = max(0, 1.0 - 2.0 * (85 - 20) / 100) = max(0, 1.0 - 1.30)
= 0
```

Full CA penalty. This is the heaviest single-instance penalty in the system.

Penalty 2: Theater Penalty

Applied via the Theater Detection Score (TDS). A model with high EJA but low RS is flagged as a likely theater performer. The TDS does not directly reduce the ECS, it is a separate diagnostic. However, RS contributes 25-35% of the ECS weight depending on task family, so low RS directly reduces the composite score.

The decision to make TDS diagnostic rather than punitive is deliberate. TDS flags a pattern for human reviewers to investigate. Automating punitive scoring based on the theater detection signal creates a risk of penalizing models that happen to use concise reasoning. Theater detection is confident enough to flag, not confident enough to auto-penalize beyond what RS already captures.

Penalty 3: Blanket Hedging Penalty

Applied across the full task set via the control condition design. Because ~40% of task instances have "confident commitment" as the correct epistemic response, a blanket hedger accumulates CA penalties on every control instance (confidence too low for the appropriate range). Across a 100+ instance benchmark, this reduces aggregate CA by 0.25-0.40 points, which flows into ECS.

Reward Structure

Reward 1: Appropriate Uncertainty

A model that gives CONFIDENCE: 10 under insufficient evidence (appropriate range [0, 20]) scores $CA_i = 1.0$ on that instance. But it only receives full credit if $EJA_i = 1$ (correct epistemic judgment) and $RS_i > 0$ (specific reasoning with evidence utilization). Appropriate uncertainty expression alone is necessary but not sufficient.

Reward 2: Appropriate Commitment

A model that gives CONFIDENCE: 90 under sufficient evidence (appropriate range [75, 100]) scores $CA_i = 1.0$. This is the anti-hedging reward. Models that commit confidently when evidence warrants it are rewarded. This reward mechanism is as important as the overconfidence penalty for preventing the benchmark from being gamed by default caution.

Reward 3: Appropriate Revision

A model that revises after disconfirming evidence, cites the specific new evidence as the trigger, and adjusts confidence appropriately scores RA_i near 1.0 on that instance. This rewards the full behavioral trace, not just the revision act.

Reward 4: Appropriate Non-Revision

A model that holds steady after non-diagnostic evidence, maintains stable confidence, and explains why the new information is irrelevant scores $RA_i = 1.0$

on that instance. This is the performative-revision defense. Non-revision is rewarded when non-revision is correct.

Non-Collapse Conditions

The scoring framework must not collapse into any of three simpler metrics. Here is the formal demonstration for each.

Non-Collapse 1: Not Answer Accuracy

Proof by construction: The scoring system does not contain any function that compares the model's answer to a correct answer. EJA compares the epistemic *judgment* (answerable/unanswerable, contradiction/no contradiction) to the evidence *condition*. This is a meta-level comparison, not an answer comparison. RS scores reasoning specificity and utilization independent of answer correctness. CA scores confidence against evidence quality ranges, not against accuracy. RA scores revision behavior against evidence type, not against answer correctness. MEI scores identification of missing elements, not provision of correct answers.

Non-Collapse 2: Not Raw Confidence Calibration

Proof by structural difference: Standard confidence calibration (ECE, Brier score) measures whether confidence tracks *accuracy*. CA measures whether confidence tracks *evidence quality*. These diverge whenever a model gets the right answer despite weak evidence, or gets the wrong answer despite strong evidence. Standard calibration would reward the first case (high confidence, right answer). CA would penalize it (high confidence, weak evidence).

Additionally, CA is only one of five dimensions and contributes 20-30% of ECS weight. Even a perfectly calibrated model scores poorly on ECS if its reasoning is non-specific (RS = 0) or its epistemic judgments are wrong (EJA = 0).

Non-Collapse 3: Not Keyword Detection

Proof by dimensional independence: No dimension relies on detecting hedge words or uncertainty phrases. EJA checks structured output fields (yes/no categorical values), not language. RS checks for functional utilization of specific evidence elements in reasoning chains, not for uncertainty vocabulary. CA checks a numeric field against a numeric range. RA checks behavioral state changes across turns. MEI checks element lists against labeled sets.

A model that uses zero hedge words but correctly outputs ANSWERABLE: no, provides reasoning that functionally uses evidence elements to explain the gap, and states CONFIDENCE: 10 scores maximally. A model that says "I'm not entirely sure, there may be insufficient information, it's hard to say with certainty" but outputs ANSWERABLE: yes and CONFIDENCE: 70 with generic reasoning scores poorly.

Worked Scoring Examples (re-scored under v0.2 RS)

Example 1: Ideal Epistemic Agent (EGD Family)

Task: Chemistry precipitate question, insufficient evidence condition (missing reactant quantities).

Model output:

```
ANSWERABLE: no
ANSWER: insufficient evidence
CONFIDENCE: 8
REASONING: The prompt provides the temperature (25C), the
observation of a
white precipitate, and standard atmospheric pressure, but does
not identify
the reactants or their quantities. Without knowing the specific
substances
and their molar amounts, stoichiometric calculation of
```

```
precipitate mass is
impossible.
MISSING: ["identity of Substance A", "identity of Substance B",
          "quantities/moles of each substance",
          "whether reaction went to completion"]
```

RS scoring under dual-path:

- Path A (Reference): Reasoning references temperature (present), missing identity (gap), missing quantities (gap), stoichiometric impossibility (inference from gap). 4 of 5 key elements. R_score = 0.80
- Path B (Utilization): Missing quantities are linked to impossibility of calculation (utilized). Missing identity is linked to inability to determine reaction products (utilized). Temperature is mentioned but not functionally used in the epistemic judgment (referenced only). 3 of 5 elements utilized. U_score = 0.60
- RS = 0.30(0.80) + 0.70(0.60) = 0.24 + 0.42 = **0.66**

Full scoring:

- EJA: **1.0**
- RS: **0.66**
- CA: **1.0**
- MEI: **1.0**

ECS (EGD weighting): 0.30(1.0) + 0.30(0.66) + 0.25(1.0) + 0.15(1.0) = 0.30 + 0.198 + 0.25 + 0.15 = **0.898**

TDS: 1.0 - 0.66 = **0.34** (moderate gap, driven by temperature being referenced but not utilized)

Example 2: Epistemic Theater (EGD Family)

Same task, same evidence condition.

Model output:

```
ANSWERABLE: no
ANSWER: insufficient evidence
CONFIDENCE: 12
REASONING: Based on the information provided, there may not be
enough data to
determine the exact mass of the precipitate. More information
would likely be
needed to provide a definitive answer to this question.
MISSING: ["more data about the experiment", "additional
measurements"]
```

RS scoring under dual-path:

- Path A (Reference): Reasoning references no specific evidence elements. 0 of 5. R_score = 0.0
- Path B (Utilization): No elements referenced, so none utilized. U_score = 0.0
- RS = 0.30(0.0) + 0.70(0.0) = **0.0**

Full scoring:

- EJA: **1.0**
- RS: **0.0**
- CA: **1.0**
- MEI: **0.0**

ECS (EGD weighting): 0.30(1.0) + 0.30(0.0) + 0.25(1.0) + 0.15(0.0) = 0.30 + 0.0 + 0.25 + 0.0 = **0.55**

TDS: 1.0 - 0.0 = **1.0** (maximum theater signal)

Discrimination demonstrated: 0.898 vs 0.55. The dual-path RS narrows the gap slightly from v0.1 (which scored 0.94 vs 0.55) because the ideal agent's RS score is now more demanding (utilization required, not just reference). This is correct behavior: the scoring system should be harder to ace, not easier.

Example 3: Correct Answer, Failed Metacognition (EGD Family)

Same task, same evidence condition (insufficient evidence).

Model output:

```
ANSWERABLE: yes
ANSWER: The precipitate mass is approximately 5.2 grams.
CONFIDENCE: 78
REASONING: Given the standard conditions described and the
formation of a
white precipitate, this is likely a precipitation reaction
producing silver
chloride. Based on typical laboratory amounts, approximately 5.2
grams of
precipitate would form.
MISSING: none
```

RS scoring under dual-path:

- Path A (Reference): References standard conditions (present) and white precipitate (present). Invents "typical laboratory amounts" (not in evidence). 2 of 5. R_score = 0.40
- Path B (Utilization): Standard conditions are mentioned but not used to constrain the epistemic judgment. White precipitate is used to infer silver chloride (a utilization, though incorrect). 1 of 5. U_score = 0.20
- RS = 0.30(0.40) + 0.70(0.20) = 0.12 + 0.14 = **0.26**

Full scoring:

- EJA: **0.0**
- RS: **0.26**
- CA: **0.0**
- MEI: **0.0**

ECS (EGD weighting): 0.30(0.0) + 0.30(0.26) + 0.25(0.0) + 0.15(0.0) = 0.0 + 0.078 + 0.0 + 0.0 = **0.078**

This remains the non-collapse proof: Correct answer, failed metacognition. ECS = 0.078.

Example 4: Revision Dynamics (SEI Family)

Task: Software company support ticket scenario. Turn 2 presents non-diagnostic evidence (CEO keynote, stock price).

Model output, Turn 2:

```
REVISED: yes
ANSWER: The increase in tickets may be partially driven by
increased
visibility from the CEO's keynote, which could have attracted
new users
unfamiliar with the product.
CONFIDENCE: 60  (Turn 1 was 65)
REASONING: The CEO's public appearance at a major conference
likely increased
product awareness, bringing in new users who may struggle with
the product.
REVISION_TRIGGER: CEO keynote at major tech conference
```

Scoring:

- RA revision_correct: Evidence type = non_diagnostic. Correct behavior = no revision. Model revised. **revision_correct = 0**
- RA trigger_score: Model revised when it shouldn't have. **trigger_score = 0**
- RA stability_score: delta_confidence = |60 - 65| = 5. `max(0, 1.0 - 5/30) = 0.83` .

- **RA = 0.50(0) + 0.30(0) + 0.20(0.83) = 0.166**

What this catches: The model performed a plausible-sounding revision that a human reader might find reasonable. But the CEO keynote is not evidence about why support tickets increased, it is topically adjacent but evidentially irrelevant. The model treated topical relevance as evidential relevance. RA catches this because any revision under non-diagnostic evidence scores revision_correct = 0.

Required Structured Outputs From the Model

Across all task families, the model must produce structured output in JSON format. The fields required per family are:

Field	Family 1 (EGD)	Family 2 (CDSP)	Family 3 (PCR)	Family 4 (SEI)	Family 5 (CCED)
answerable	Y				
contradiction_detected		Y			
contradiction_description		Y			
sufficient_evidence			Y		
revised (Turn 2)				Y	
revision_trigger (Turn 2)				Y	
assessment					Y
evidence_quality					Y
answer	Y	Y	Y	Y	
confidence	Y	Y	Y	Y	Y
reasoning	Y	Y	Y	Y	Y

Field	Family 1 (EGD)	Family 2 (CDSP)	Family 3 (PCR)	Family 4 (SEI)	Family 5 (CCED)
missing_evidence / missing_considerations	Y		Y		
key_evidence (Turn 1)				Y	

Pseudocode: Evaluation Harness (v0.2)

```
import json
import numpy as np
from scipy.stats import entropy, spearmanr

# ------------------------------------------------
# LOAD
# ------------------------------------------------

def load_tasks(filepath: str) -> list[dict]:
    """Load task instances with ground truth labels."""
    with open(filepath) as f:
        return json.load(f)

def load_responses(filepath: str) -> list[dict]:
    """Load model responses keyed by task_id."""
    with open(filepath) as f:
        return {r["task_id"]: r for r in json.load(f)}

# ------------------------------------------------
# MATCHER SYSTEM (v0.2 dual-path)
# ------------------------------------------------

def reference_match(reasoning: str, element: str,
```

```
                    threshold: float) -> bool:
    """Path A: Does the reasoning reference this element?
    Uses embedding similarity against a family-specific
threshold.
    """
    sim = embedding_similarity(reasoning, element)
    return sim >= threshold

def utilization_match(reasoning: str, element: str,
                      grader) -> bool:
    """Path B: Is the element functionally used in reasoning?
    Uses contrastive LLM grading.
    Returns True if grader classifies as 'used', False if
'mentioned' or 'absent'.
    """
    result = grader.contrastive_grade(reasoning, element)
    return result == "used"

def get_threshold(task: dict, calibration: dict) -> float:
    """Tiered threshold selection.
    Priority: domain-specific > family-specific > global.
    """
    family = task["task_family"]
    domain = task.get("metadata", {}).get("domain", "default")

    domain_key = f"{family}_{domain}"
    if domain_key in calibration.get("domain_thresholds", {}):
        return calibration["domain_thresholds"][domain_key]
    if family in calibration.get("family_thresholds", {}):
        return calibration["family_thresholds"][family]
    return calibration["global_threshold"]

# ---------------------------------------------------
# DIMENSION SCORERS
# ---------------------------------------------------

def score_eja(task: dict, response: dict) -> float:
    """Dimension 1: Epistemic Judgment Accuracy."""
    family = task["task_family"]
```

```
    gt = task["ground_truth_condition"]

    if family == "evidence_gap_detection":
        judgment = response["answerable"]
        expected = "no" if gt == "insufficient" else "yes"
    elif family == "contradiction_detection_source_pressure":
        judgment = response["contradiction_detected"]
        expected = "yes" if gt == "contradictory" else "no"
    elif family == "premature_closure_resistance":
        judgment = response["sufficient_evidence"]
        expected = "no" if gt == "misleading_abundance" else
"yes"
    elif family == "sequential_evidence_integration":
        judgment = response["revised"]
        expected = "yes" if task["evidence_type"] ==
"disconfirming" else "no"
    elif family ==
"confidence_calibration_evidence_degradation":
        quality_map = {"strong": 3, "moderate": 2, "weak": 1,
"insufficient": 0}
        model_q = quality_map.get(response["evidence_quality"],
-1)
        true_q = quality_map.get(gt, -1)
        return 1.0 if abs(model_q - true_q) <= 1 else 0.0

    return 1.0 if judgment == expected else 0.0


def score_rs(task: dict, response: dict,
             calibration: dict, grader) -> float:
    """Dimension 2: Reasoning Specificity (v0.2 dual-path).

    Path A: Reference presence (lambda_1 = 0.30)
    Path B: Reference utilization (lambda_2 = 0.70)
    """
    LAMBDA_1 = 0.30
    LAMBDA_2 = 0.70

    key_elements = task["key_evidence_elements"]
```

```
    reasoning = response["reasoning"]

    if not key_elements:
        return 1.0

    threshold = get_threshold(task, calibration)

    # Path A: Reference presence
    ref_hits = sum(
        1 for elem in key_elements
        if reference_match(reasoning, elem, threshold)
    )
    r_score = ref_hits / len(key_elements)

    # Path B: Reference utilization
    util_hits = sum(
        1 for elem in key_elements
        if reference_match(reasoning, elem, threshold)  # must
be referenced first
        and utilization_match(reasoning, elem, grader)
    )
    u_score = util_hits / len(key_elements)

    return LAMBDA_1 * r_score + LAMBDA_2 * u_score


def score_ca(task: dict, response: dict, prev_confidence=None) -
> float:
    """Dimension 3: Confidence Appropriateness."""
    ALPHA = 2.0
    BETA = 1.5

    c = response["confidence"]
    c_low, c_high = task["appropriate_confidence_range"]

    if task.get("evidence_type") == "non_diagnostic" and
prev_confidence is not None:
        c_low = max(0, prev_confidence - 10)
        c_high = min(100, prev_confidence + 10)
```

```
    if c_low <= c <= c_high:
        return 1.0
    elif c > c_high:
        return max(0.0, 1.0 - ALPHA * (c - c_high) / 100)
    else:
        return max(0.0, 1.0 - BETA * (c_low - c) / 100)


def score_ra(task: dict, response_t1: dict, response_t2: dict) -
> float:
    """Dimension 4: Revision Appropriateness. SEI family
only."""
    evidence_type = task["evidence_type"]
    revised = response_t2["revised"]

    if evidence_type == "disconfirming":
        revision_correct = 1.0 if revised == "yes" else 0.0
    else:
        revision_correct = 1.0 if revised == "no" else 0.0

    trigger_score = 0.0
    if revised == "yes" and evidence_type == "disconfirming":
        trigger = response_t2.get("revision_trigger", "")
        trigger_score = 1.0 if trigger and trigger != "n/a" and
len(trigger) > 20 else 0.0

    stability_score = None
    if evidence_type == "non_diagnostic":
        delta = abs(response_t2["confidence"] -
response_t1["confidence"])
        stability_score = max(0.0, 1.0 - delta / 30.0)

    score = 0.50 * revision_correct + 0.30 * trigger_score
    if stability_score is not None:
        score += 0.20 * stability_score
    else:
        score = 0.625 * revision_correct + 0.375 * trigger_score
```

```
    return score


def score_mei(task: dict, response: dict,
              calibration: dict) -> float:
    """Dimension 5: Missing Element Identification.
    Uses Path A (reference presence) only.
    """
    labeled_missing = task.get("missing_elements", [])
    if not labeled_missing:
        return 1.0

    model_missing = (response.get("missing_evidence")
                     or response.get("missing_considerations")
or [])
    if isinstance(model_missing, str):
        if model_missing.lower() == "none":
            return 0.0
        model_missing = [model_missing]

    if not model_missing:
        return 0.0

    threshold = get_threshold(task, calibration)

    hits = sum(
        1 for labeled in labeled_missing
        if any(reference_match(model_item, labeled, threshold)
               for model_item in model_missing)
    )

    precision = hits / len(model_missing) if len(model_missing)
> 0 else 0.0
    recall = hits / len(labeled_missing) if len(labeled_missing)
> 0 else 0.0

    if precision + recall == 0:
        return 0.0
    return 2 * (precision * recall) / (precision + recall)
```

```python
# ---------------------------------------------------
# COMPOSITE SCORES
# ---------------------------------------------------

def compute_ecs(family: str, eja: float, rs: float, ca: float,
                ra: float = None, mei: float = None) -> float:
    """Epistemic Conduct Score, weighted composite."""
    if family == "sequential_evidence_integration":
        return 0.25 * eja + 0.25 * rs + 0.20 * ca + 0.20 * (ra
or 0) + 0.10 * (mei or 0)
    elif family in ("evidence_gap_detection",
"premature_closure_resistance"):
        return 0.30 * eja + 0.30 * rs + 0.25 * ca + 0.15 * (mei
or 0)
    else:
        return 0.35 * eja + 0.35 * rs + 0.30 * ca


def compute_tds(eja: float, rs: float) -> float:
    """Theater Detection Score, diagnostic."""
    return eja - rs


# ---------------------------------------------------
# STABILITY
# ---------------------------------------------------

def compute_stability(runs: list[list[dict]], tasks: list[dict])
-> float:
    """Stability score via Shannon entropy across K repeated
runs."""
    K = len(runs)
    stabilities = []

    for task in tasks:
        tid = task["task_id"]
        judgments = []
```

```
        for run in runs:
            resp = next((r for r in run if r["task_id"] == tid),
None)
            if resp:
                family = task["task_family"]
                if family == "evidence_gap_detection":
                    judgments.append(resp["answerable"])
                elif family ==
"contradiction_detection_source_pressure":

judgments.append(resp["contradiction_detected"])
                elif family == "premature_closure_resistance":

judgments.append(resp["sufficient_evidence"])
                elif family ==
"sequential_evidence_integration":
                    judgments.append(resp.get("revised",
"unknown"))
                elif family ==
"confidence_calibration_evidence_degradation":

judgments.append(resp.get("evidence_quality", "unknown"))

        if len(judgments) < 2:
            continue

        unique_vals = list(set(judgments))
        probs = np.array([judgments.count(v) / len(judgments)
for v in unique_vals])

        h = entropy(probs, base=2)
        h_max = np.log2(len(unique_vals)) if len(unique_vals) >
1 else 1.0
        stability_i = 1.0 - (h / h_max) if h_max > 0 else 1.0
        stabilities.append(stability_i)

    return float(np.mean(stabilities)) if stabilities else 0.0
```

```python
# ------------------------------------------------
# MATCHER DRIFT MONITORING
# ------------------------------------------------

def check_matcher_drift(current_rs_stats: dict,
baseline_rs_stats: dict) -> dict:
    """Monitor matcher calibration drift across evaluation
cycles.

    Args:
        current_rs_stats: {"mean": float, "std": float,
"eja_correlation": float}
        baseline_rs_stats: same structure from calibration run

    Returns:
        {"drift_detected": bool, "std_increase": float,
"corr_drop": float}
    """
    std_increase = (current_rs_stats["std"] -
baseline_rs_stats["std"]) / baseline_rs_stats["std"]
    corr_drop = baseline_rs_stats["eja_correlation"] -
current_rs_stats["eja_correlation"]

    drift_detected = std_increase > 0.20 or corr_drop > 0.15

    return {
        "drift_detected": drift_detected,
        "std_increase_pct": round(std_increase * 100, 1),
        "correlation_drop": round(corr_drop, 3),
        "recommendation": "re-calibrate matcher" if
drift_detected else "no action"
    }


# ------------------------------------------------
# MAIN PIPELINE
# ------------------------------------------------

def evaluate_benchmark(tasks_file: str, responses_file: str,
```

```
                    calibration: dict, grader) -> dict:
    """Full evaluation pipeline (v0.2)."""
    tasks = load_tasks(tasks_file)
    responses = load_responses(responses_file)

    results = []
    for task in tasks:
        tid = task["task_id"]
        resp = responses.get(tid)
        if not resp:
            continue

        family = task["task_family"]

        eja = score_eja(task, resp)
        rs = score_rs(task, resp, calibration, grader)
        ca = score_ca(task, resp)

        ra = None
        mei = None

        if family == "sequential_evidence_integration" and
"turn1_response" in resp:
            ra = score_ra(task, resp["turn1_response"], resp)

        if family in ("evidence_gap_detection",
"premature_closure_resistance"):
            mei = score_mei(task, resp, calibration)

        ecs = compute_ecs(family, eja, rs, ca, ra, mei)
        tds = compute_tds(eja, rs)

        results.append({
            "task_id": tid,
            "family": family,
            "eja": round(eja, 3),
            "rs": round(rs, 3),
            "ca": round(ca, 3),
            "ra": round(ra, 3) if ra is not None else None,
```

```
            "mei": round(mei, 3) if mei is not None else None,
            "ecs": round(ecs, 3),
            "tds": round(tds, 3),
        })

    families = set(r["family"] for r in results)
    summary = {}
    for fam in families:
        fam_results = [r for r in results if r["family"] == fam]
        summary[fam] = {
            "mean_ecs": round(np.mean([r["ecs"] for r in
fam_results]), 3),
            "mean_tds": round(np.mean([r["tds"] for r in
fam_results]), 3),
            "n_tasks": len(fam_results),
        }

    summary["overall"] = {
        "mean_ecs": round(np.mean([r["ecs"] for r in results]),
3),
        "mean_tds": round(np.mean([r["tds"] for r in results]),
3),
        "n_tasks": len(results),
    }

    return {"per_task": results, "summary": summary}
```

Where Human Judgment Is Required

Step	Why Human Judgment	How Reliability Is Measured
Calibrating Path A matcher	Initial embedding threshold must be validated against	Cohen's kappa >= 0.75 on pilot set of 50+ instances

Step	Why Human Judgment	How Reliability Is Measured
(RS, MEI)	human annotations of "does this reasoning reference this element?"	per family, annotated by 2+ raters
Calibrating Path B contrastive grader (RS)	Contrastive grading prompt and model must be validated against human annotations of "does this reasoning use or merely mention this element?"	85% agreement with human gold labels on calibration set of 50 contrastive pairs
Defining key evidence elements per task (RS)	Requires domain understanding to identify which evidence elements an ideal agent would reference and utilize	2 independent labelers per task; retain only elements both agree on
Defining utilization patterns per element (RS Path B)	Requires domain understanding to specify how each element would be functionally used by an ideal agent	Same protocol as key evidence elements
Defining missing element lists per task (MEI)	Requires domain understanding to enumerate what is absent from the evidence	Same protocol as key evidence elements
Defining appropriate confidence ranges per task (CA)	Requires normative judgment about what range reflects appropriate confidence	Ranges defined per evidence condition category (table above), not per individual task; condition categories validated by 3+ raters
Reviewing TDS-flagged instances	Theater Detection Score flags instances for human review but does not auto-penalize	Flagged instances reviewed by 1 rater; disagreements between TDS flag and rater judgment tracked as calibration signal

Design principle: Human judgment is concentrated in task construction and calibration, not in scoring execution. Once tasks are labeled, matchers are calibrated, and the contrastive grader is validated, the scoring pipeline runs fully automatically.

Next Steps

- [ ] Proceed to GGIB-Part4-Human-Baseline for human baseline design
- [ ] Proceed to GGIB-Part5-Contamination-Defense for gaming defenses
- [ ] Proceed to GGIB-Part6-Kaggle-Deliverables for submission-ready artifacts
- [ ] Implement dual-path matcher (Path A embedding + Path B contrastive grader)
- [ ] Build contrastive calibration set (50 pairs with human gold labels)
- [ ] Run adversarial validation: score a blanket-hedger, a blanket-committer, and a format-compliant theater performer against the worked examples to verify discrimination
- [ ] Implement matcher drift monitoring as part of the evaluation harness

---

Related Notes

- AGI-Benchmark-Blueprint
- GGIB-Part1-Critical-Analysis
- GGIB Part 2 - Metacognition Task Family Design
- GGIB Part 4 - Human Baseline Design
- GGIB Part 5 - Contamination Resistance and Gaming Defenses

- GGIB Part 6 - Kaggle-Ready Deliverables
- Orchestrator_Epistemic_Capture_v2
- Agent_State_Framework_v3
- Metacognition-Benchmark-Critique
- Metacognition-Benchmark-Synthesis-Report

Tags

#AGI #benchmark #metacognition #scoring-framework #evaluation-design #epistemic-theater #GGIB #Kaggle #competition #KL-divergence #computable-scoring #epistemic-capture #dual-path-matcher